République du Sénégal

Un Peuple – Un But – Une Foi

Ministère de l'Enseignement Supérieur et de la Recherche

Université Alioune DIOP de Bambey

UFR Sciences Appliquées et Technologies de l'Information et de la Communication **(SATIC)**

Département Technologies de l'Information et de la Communication **(TIC)**

Master professionnelle

Systèmes D'Information **(S.I)**

Déploiement d'une Architecture Réseau 3-Tiers Virtualisée sur OpenShift

Encadrant(s) :

Dr S. S. M. BABOU

Présenté par :

Adama Seck

Année académique 2025-2026

Table des matières

Contexte et objectifs 6

Architecture finale 6

Étapes détaillées de réalisation 8

1. Nettoyage initial de l'environnement 8

2. Déploiement de MySQL avec persistance 8

3. Création des ConfigMaps pour les applications 11

server.js (node-api-code) 11

Python (python-api-code) 13

Frontend (ConfigMap frontend-files avec index.html, style.css, script.js) 15

4. Déploiement des APIs et du frontend 17

Node.js 17

Python 18

Frontend 18

5. Résolution des problèmes de connectivité réseau 19

a) Communication interne bloquée → NetworkPolicy allow-all-internal 19

b) Routes externes renvoyant 503 → correction des sélecteurs de services et ajout de allow-from-openshift-ingress 20

c) Dernier recours : utilisation de HTTP (sans TLS) pour les routes 21

6. Mise en place de GitHub Actions 23

Création d'un compte de service et d'un token OpenShift 23

Secrets GitHub 24

Workflow .github/workflows/deploy.yml 25

Problèmes rencontrés et solutions 28

État final fonctionnel 29

Automatisation CI/CD (GitHub Actions) 31

Commandes utiles (annexe) 33
Gestion des ressources 33
Redémarrage et logs 33
NetworkPolicy 33
Port-forward (pour test local) 34
GitHub Actions – mise à jour du token 34
Conclusion 34
Lien gituhp du projet : https://github.com/Adam01-i/my-webapp 34
Lien public de l'application : http://web-frontend-route-adam01-i-dev.apps.rm3.7wse.p1.openshiftapps.com/ 34

Index des figures

Figure 1: Schéma des communications 7
Figure 2: PVC : mysql-pvc 9
Figure 3: Deploiement : mysql 10
Figure 4: Service : mysql 11
Figure 5: ConfigMaps pour les applications : server.js (node-api-code) 13
Figure 6: ConfigMaps pour les applications : Python (python-api-code) 14
Figure 7: ConfigMaps pour les applications : index.html 15
Figure 8: ConfigMaps pour les applications : style.css 16
Figure 9: ConfigMaps pour les applications : script.js 17
Figure 10: Resultats des déploiement des APIs et du frontend 19
Figure 11: NetworkPolicy : allow-all-internal 20
Figure 12: NetworkPolicy : allow-from-openshift-ingress 21
Figure 13: Utilisation de HTTP (sans TLS) pour les routes : node-route 22
Figure 14: Utilisation de HTTP (sans TLS) pour les routes : python-route 22
Figure 15: Utilisation de HTTP (sans TLS) pour les routes : web-frontend-route 23
Figure 16: Création d'un compte de service et d'un token OpenShift 24
Figure 17: Secrets GitHub 25
Figure 18: Workflow .github/workflows/deploy.yml 26
Figure 19: Resultats des builds declenches : img1 27
Figure 20: Resultats des builds declenches : img2 27
Figure 21: Resultats des builds declenches : img3 28
Figure 22: Tests curl : routes : python, node, frontend 30
Figure 23: Resultat du lien public de l'app via Interface web 31
Figure 24: Structure du code sur github 32

Figure 25: Resultats des builds apres un push qui declenche un deploiement automatique 33

RAPPORT COMPLET – DÉPLOIEMENT D'UNE APPLICATION WEB SUR OPENSHIFT AVEC CI/CD

Contexte et objectifs

Dans le cadre d'un projet pratique, l'objectif était de déployer une application web complète sur un **cluster OpenShift en mode sandbox**, avec les contraintes suivantes :

- Utilisation de **deux machines virtuelles (KubeVirt)** initialement, puis abandon de celles-ci au profit de pods classiques pour des raisons de stabilité.
- Mise en place d'une **base de données MySQL persistante**.
- Développement de **deux API** (Node.js et Python) communiquant avec MySQL.
- Création d'un **frontend statique** (HTML/CSS/JS) pour afficher un tableau de bord interrogeant les deux API.
- Mise en place d'une **intégration continue / déploiement continu (CI/CD)** via **GitHub Actions** : à chaque `git push`, les applications sont automatiquement reconstruites et redéployées sur OpenShift.

Ce rapport détaille l'ensemble des actions menées, des problèmes rencontrés et des solutions adoptées, en s'appuyant sur l'historique complet de la session de travail.

Architecture finale

Composant	Technologie	Accès	Persistance
Base de données	MySQL 8	Service interne `mysql-service:3306`	PVC `mysql-pvc` (5Gi)
API Node.js	Node.js 22 + `mysql2`	Route HTTP `node-route` (port 3000)	–
API Python	Flask + `pymysql` + `flask_cors`	Route HTTP `python-route` (port 8080)	–
Frontend	Nginx (`nginxinc/nginx-unprivileged`)	Route HTTP `web-frontend-route` (port 80)	–
CI/CD	GitHub Actions	Workflow sur push `main`	–

Schéma des communications :

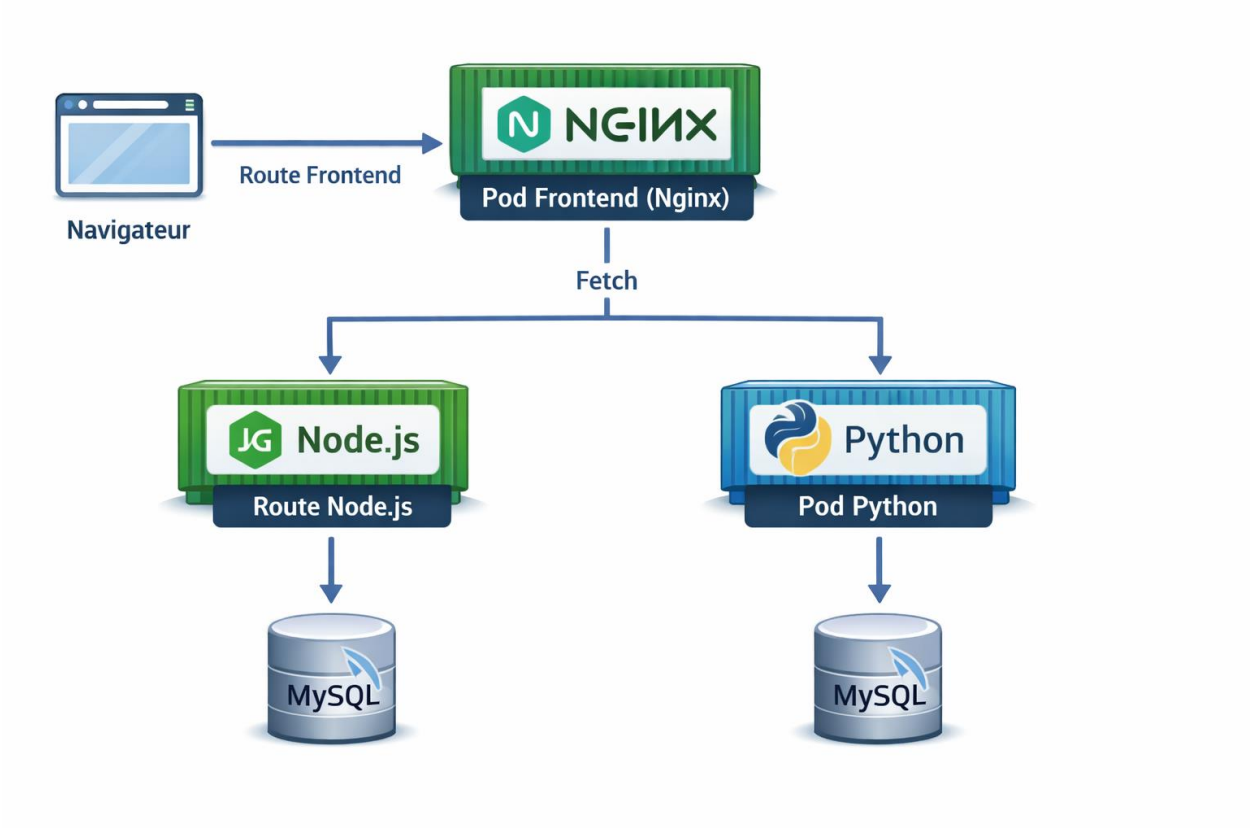

Figure 1: Schéma des communications

NetworkPolicies mises en place :

- `allow-all-internal` : autorise toutes les communications entre pods du même namespace.

- `allow-from-openshift-ingress` : autorise le trafic entrant depuis le namespace du routeur OpenShift (non fonctionnel à cause des restrictions du sandbox, mais conservée).

- `allow-all-ingress` : autorise tout trafic entrant (utilisée pour les tests, finalement la route HTTP a fonctionné sans TLS).

Étapes détaillées de réalisation

1. Nettoyage initial de l'environnement

Au début, le namespace `adam01-i-dev` contenait des ressources issues de précédents essais (VMs, pods, services, routes, configmaps). Nous avons procédé à un nettoyage complet :

```
oc delete route firewall-route nginx-route web-route
oc delete svc firewall-service nginx-service mysql-service web-service
```

```
oc delete deployment firewall-nginx web-backend
oc delete bc web-backend-custom
oc delete is web-backend-custom
oc delete configmap nginx-firewall-config
oc delete vm fedora-gray-roundworm-62--vm-bd-mysql fedora-lime-cobra-86--vm-web
oc delete dv fedora-gray-roundworm-62--vm-bd-mysql fedora-lime-cobra-86--vm-web
```

Seuls les services système (`headless`, `modelmesh-serving`, `nodejs-ex`, etc.) et le workspace de l'utilisateur ont été conservés.

2. Déploiement de MySQL avec persistance

Création du PVC :

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
 name: mysql-pvc
spec:
 accessModes: [ReadWriteOnce]
 resources:
 requests:
 storage: 5Gi
```

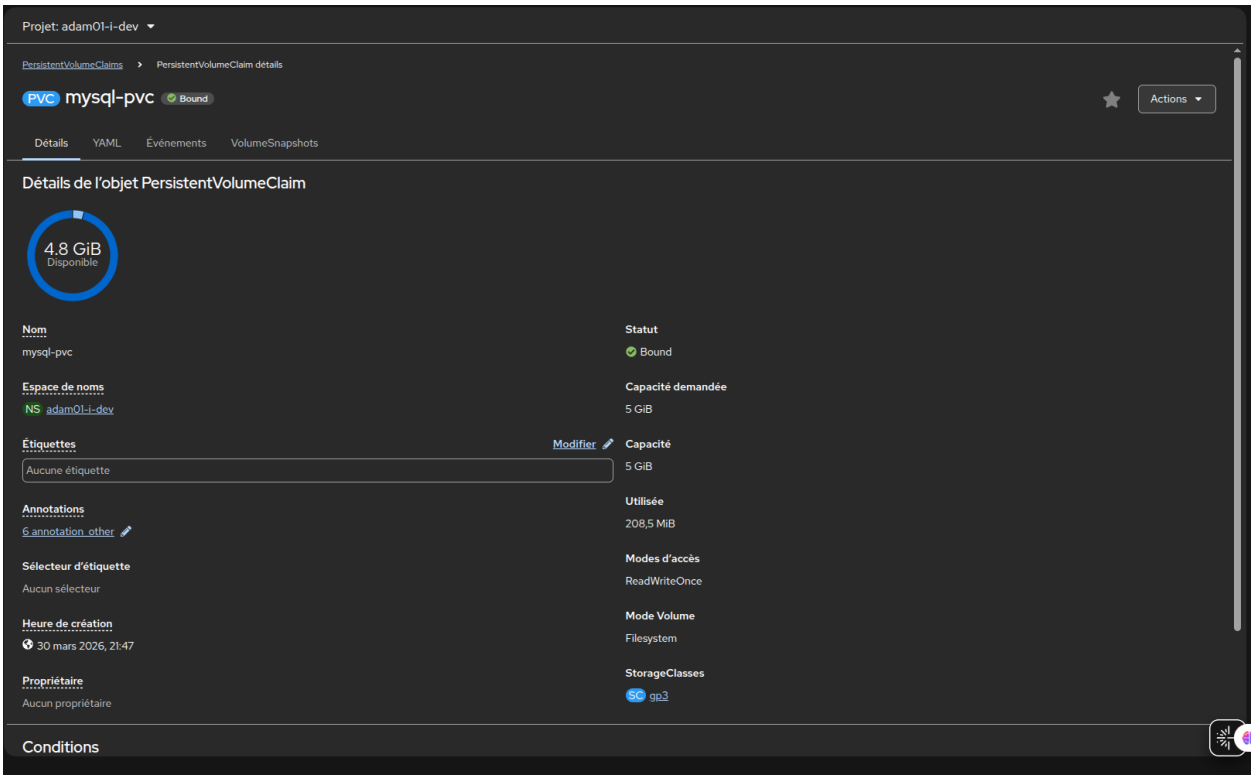

Figure 2: PVC : mysql-pvc

Déploiement MySQL :

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
 name: mysql
spec:
 replicas: 1
 selector:
 matchLabels:
 app: mysql
 template:
 metadata:
 labels:
 app: mysql
 spec:
 containers:
 - name: mysql
 image: mysql:8
 env:
 - name: MYSQL_ROOT_PASSWORD
```

```
value: rootpass
        - name: MYSQL_DATABASE
          value: webdb
        - name: MYSQL_USER
          value: webuser
        - name: MYSQL_PASSWORD
          value: webpass_
        ports:
        - containerPort: 3306
        volumeMounts:
        - name: mysql-storage
          mountPath: /var/lib/mysql
      volumes:
      - name: mysql-storage
        persistentVolumeClaim:
          claimName: mysql-pvc
```

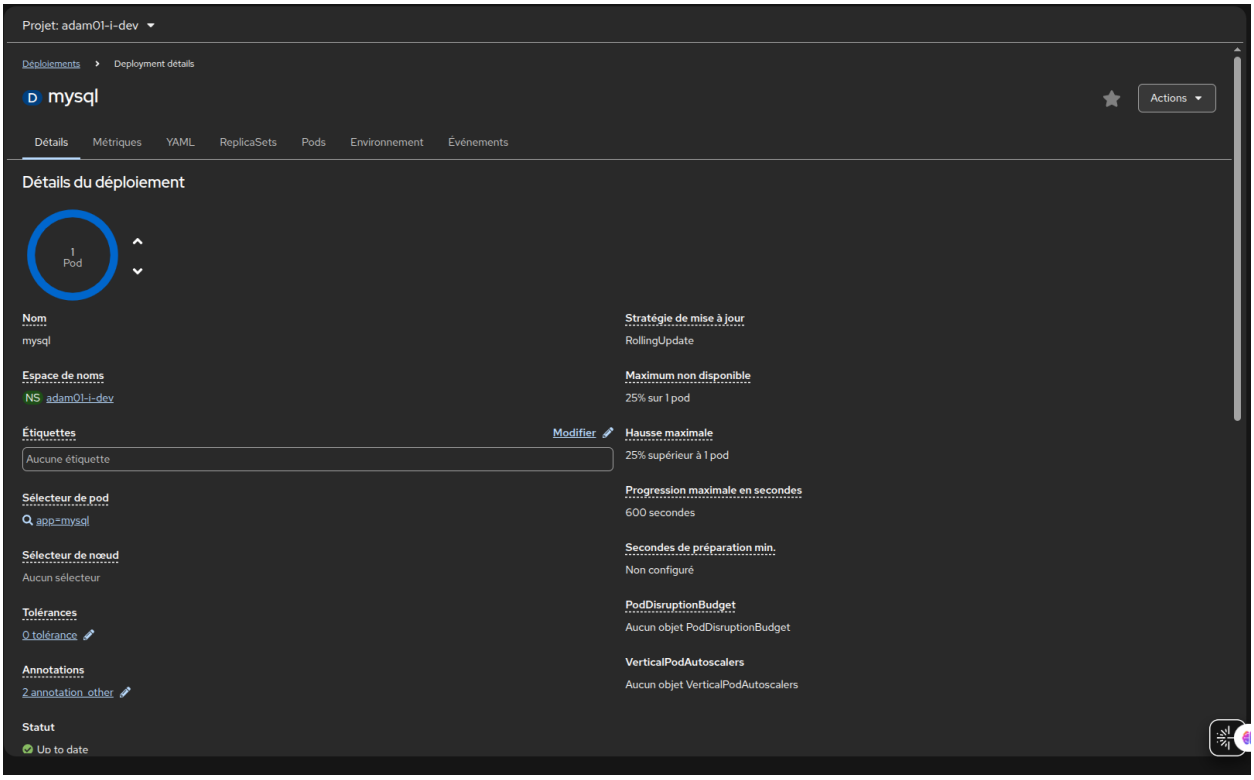

Figure 3: Deploiement : mysql

Service MySQL :

```
oc expose deployment mysql --name=mysql-service –port=3306
```

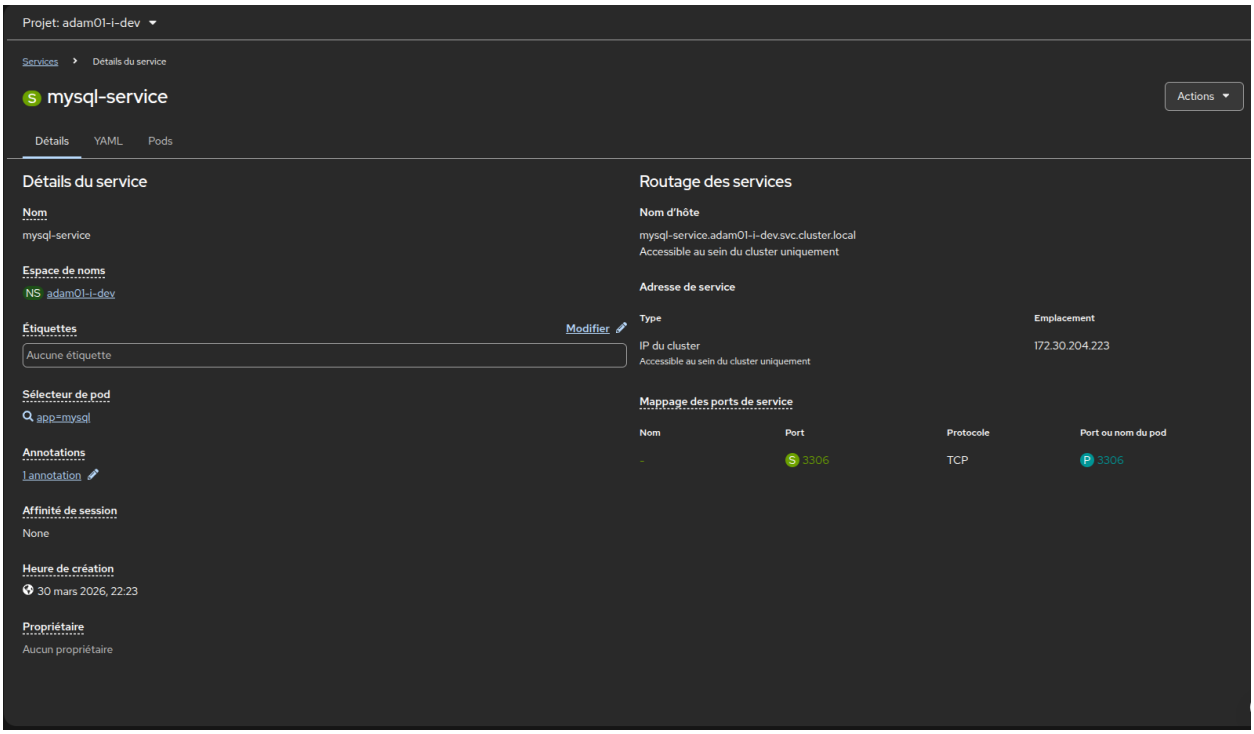

Figure 4: Service : mysql

3. Création des ConfigMaps pour les applications

server.js (node-api-code)

```
const mysql = require('mysql2');
const http = require('http');
const connection = mysql.createConnection({
 host: 'mysql-service.adam01-i-dev.svc.cluster.local',
 user: 'webuser',
 password: 'webpass_',
 database: 'webdb',
 port: 3306
});
const server = http.createServer((req, res) => {
 res.setHeader("Access-Control-Allow-Origin", "*");
 res.setHeader("Access-Control-Allow-Methods", "GET, OPTIONS");
 res.setHeader("Access-Control-Allow-Headers", "Content-Type");
 if (req.method === "OPTIONS") { res.writeHead(204); res.end(); return; }
 connection.connect(err => {
 if (err) {
 res.writeHead(500, {'Content-Type': 'application/json'});
 res.end(JSON.stringify({error: "Erreur connexion DB"}));
 return;
```

```
 }
        connection.query('SELECT "Connexion Node.js -> MySQL OK" as message',
(err, results) => {
            if (err) {
                res.writeHead(500);
                res.end(JSON.stringify({error: err.message}));
                return;
            }
            res.writeHead(200, {'Content-Type': 'application/json'});
            res.end(JSON.stringify(results[0]));
        });
    });
});
server.listen(3000, "0.0.0.0", () => console.log("API running on port
3000"));
```

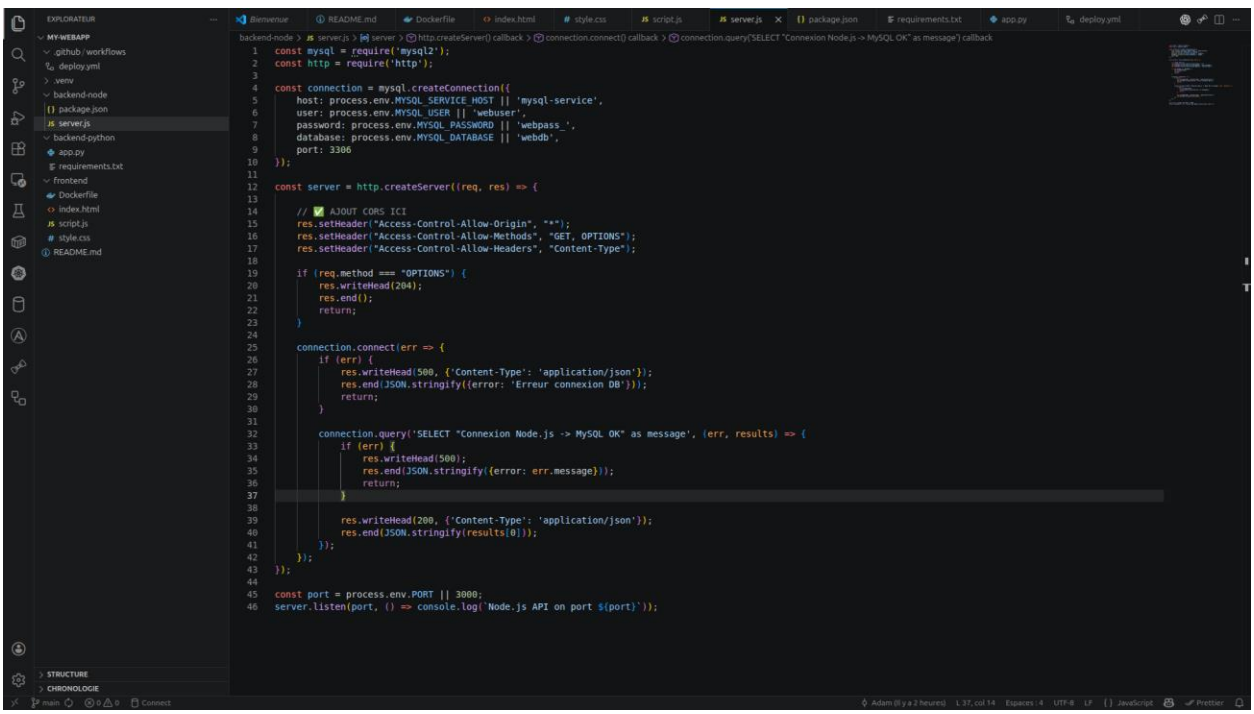

Figure 5: ConfigMaps pour les applications : server.js (`node-api-code`)

Python (python-api-code)

```
import os
import pymysql
from flask import Flask, jsonify
from flask_cors import CORS

app = Flask(__name__)
```

```python
CORS(app)

def get_db_connection():
    return pymysql.connect(
        host='mysql-service.adam01-i-dev.svc.cluster.local',
        user='webuser',
        password='webpass_',
        database='webdb',
        port=3306,
        cursorclass=pymysql.cursors.DictCursor
    )

@app.route('/')
def index():
    try:
        conn = get_db_connection()
        with conn.cursor() as cursor:
            cursor.execute("SELECT 'Connexion Python -> MySQL OK' as
message")
            result = cursor.fetchone()
        conn.close()
        return jsonify(result)
    except Exception as e:
        return jsonify({"error": str(e)}), 500

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=8080)
```

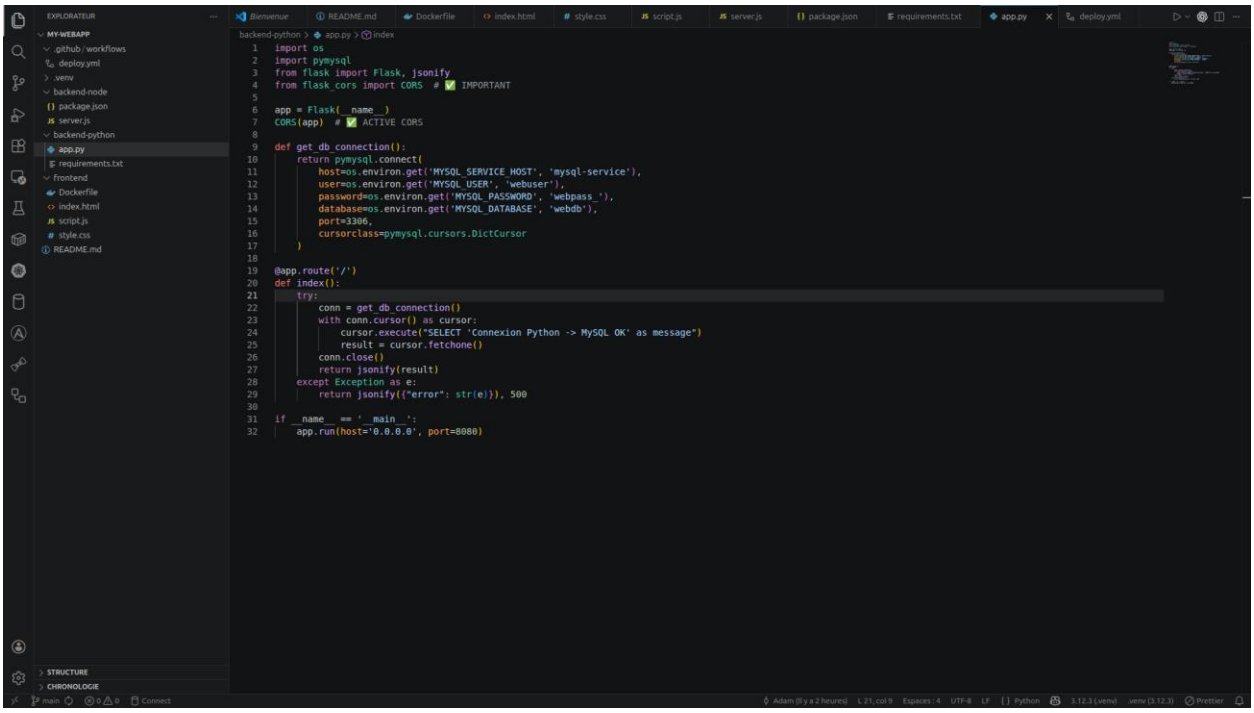

Figure 6: ConfigMaps pour les applications : Python (`python-api-code`)

Frontend (ConfigMap `frontend-files` avec `index.html`, `style.css`, `script.js`)

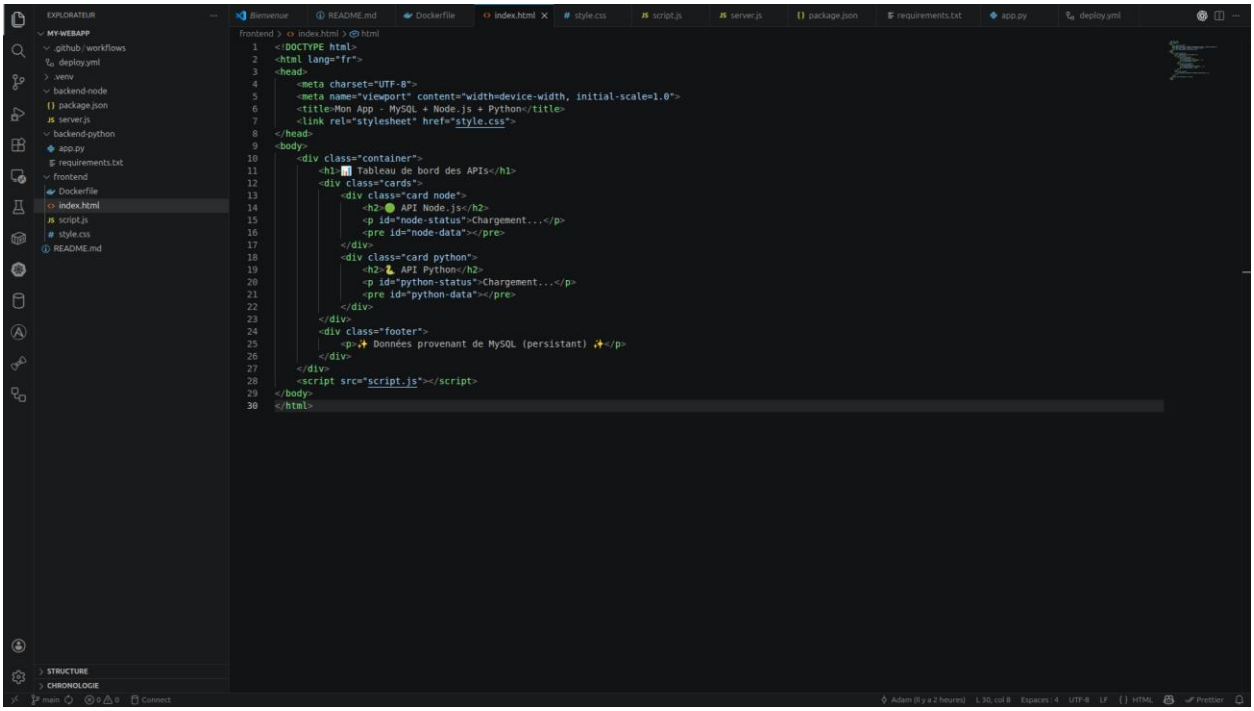

Figure 7: ConfigMaps pour les applications : `index.html`

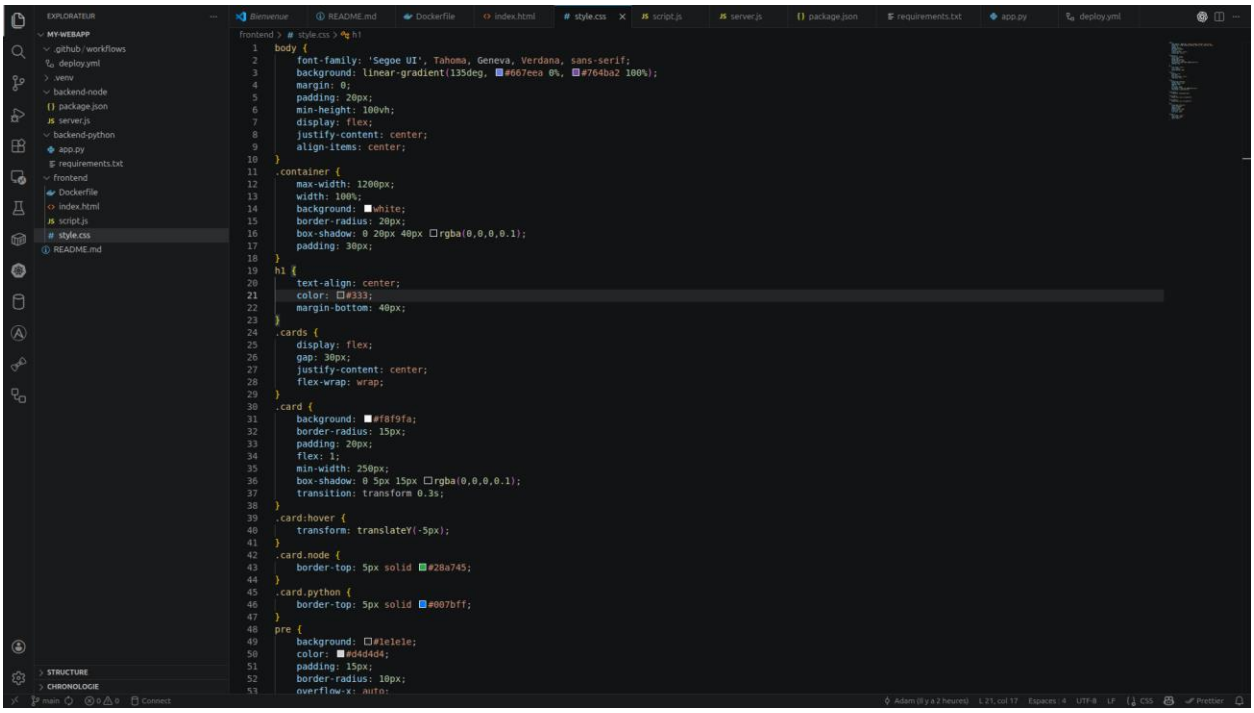

Figure 8: ConfigMaps pour les applications : `style.css`

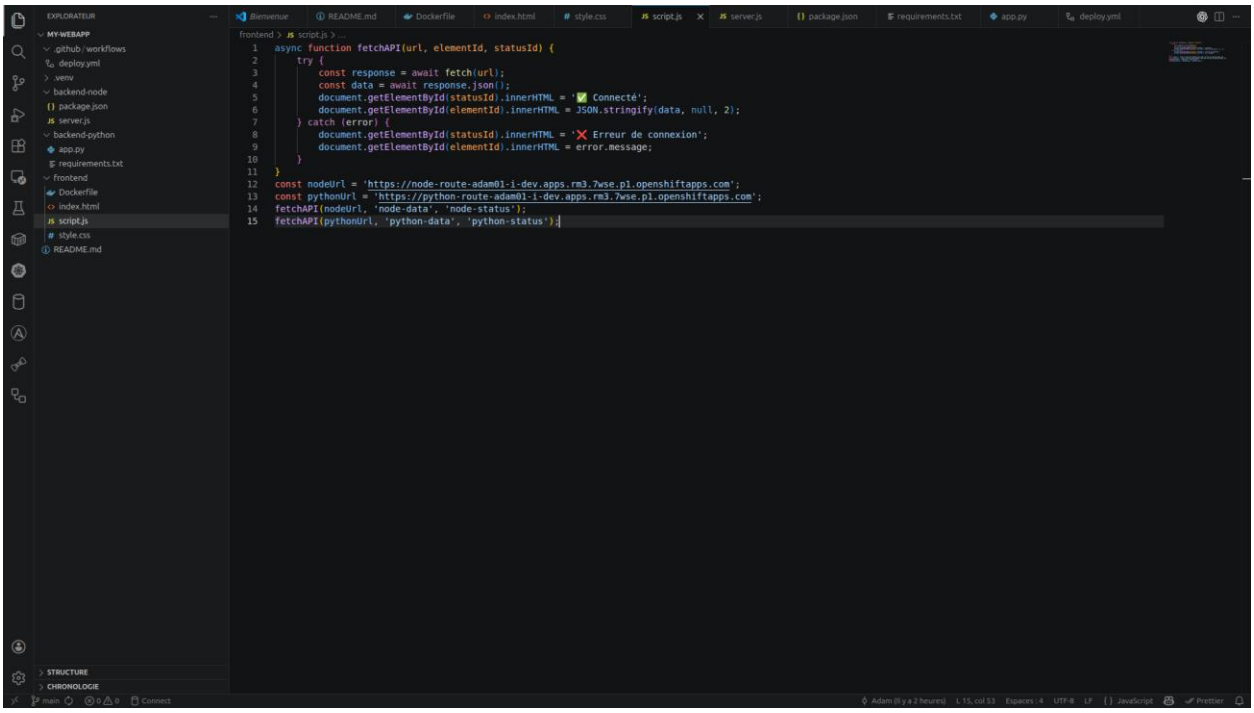

Figure 9: ConfigMaps pour les applications : script.js

Le `script.js` final utilise les URLs HTTP des routes :

```
const nodeUrl = "http://node-route-adam01-i-dev.apps.rm3.7wse.p1.openshiftapps.com";
const pythonUrl = "http://python-route-adam01-i-dev.apps.rm3.7wse.p1.openshiftapps.com";


fetchAPI(nodeUrl, "node-data", "node-status");
fetchAPI(pythonUrl, "python-data", "python-status");
```

4. Déploiement des APIs et du frontend

Node.js

```
# Création du déploiement à partir du ConfigMap (via image S2I)
oc new-app https://github.com/Adam01-i/my-webapp --context-dir=backend-node --name=node-api
oc expose svc node-api --name=node-route
# Ajout des en-têtes CORS (déjà dans le ConfigMap)
```

Python

```
oc new-app https://github.com/Adam01-i/my-webapp --context-dir=backend-python --name=python-api
oc expose svc python-api --name=python-route
```

```
Frontend
# Création du ConfigMap à partir des fichiers locaux
oc create configmap frontend-files --from-file=index.html --from-
file=style.css --from-file=script.js
# Déploiement avec nginx-unprivileged
cat <<EOF | oc apply -f -
apiVersion: apps/v1
kind: Deployment
metadata:
 name: web-frontend
spec:
 replicas: 1
 selector:
 matchLabels:
 app: web-frontend
 template:
 metadata:
 labels:
 app: web-frontend
 spec:
 containers:
 - name: nginx
 image: nginxinc/nginx-unprivileged
 ports:
 - containerPort: 8080
 volumeMounts:
 - name: frontend-files
 mountPath: /usr/share/nginx/html
 volumes:
 - name: frontend-files
 configMap:
 name: frontend-files
EOF
oc expose deployment web-frontend --port=8080 --target-port=8080 –name=web-
frontend-service

oc expose svc web-frontend-service –name=web-frontend-route
```

Voici les resultats :

Terminal de ligne de commande OpenShift

Terminal 4 +

```
bash-5.1 ~ $ oc get deployment node-api python-api web-frontend
NAME           READY   UP-TO-DATE   AVAILABLE   AGE
node-api       1/1     1            1           89m
python-api     1/1     1            1           89m
web-frontend   1/1     1            1           124m
bash-5.1 ~ $ oc get svc node-api python-api web-frontend-service
NAME                  TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)    AGE
node-api              ClusterIP   172.30.186.170   <none>        3000/TCP   61m
python-api            ClusterIP   172.30.228.99    <none>        8080/TCP   61m
web-frontend-service  ClusterIP   172.30.205.37    <none>        8080/TCP   124m
bash-5.1 ~ $ oc get route node-route python-route web-frontend-route
NAME                 HOST/PORT                                                    PATH   SERVICES               PORT   TERMINATION   WILDCARD
node-route           node-route-adam01-i-dev.apps.rm3.7wse.p1.openshiftapps.com          node-api               3000                 None
python-route         python-route-adam01-i-dev.apps.rm3.7wse.p1.openshiftapps.com        python-api             8080                 None
web-frontend-route   web-frontend-route-adam01-i-dev.apps.rm3.7wse.p1.openshiftapps.com  web-frontend-service   8080                 None
bash-5.1 ~ $ oc get configmap frontend-files
NAME             DATA   AGE
frontend-files   3      129m
bash-5.1 ~ $
bash-5.1 ~ $
bash-5.1 ~ $
```

Figure 10: Resultats des déploiement des APIs et du frontend

5. Résolution des problèmes de connectivité réseau

a) Communication interne bloquée → NetworkPolicy `allow-all-internal`

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
 name: allow-all-internal
spec:
 podSelector: {}
 ingress:
 - from:
 - podSelector: {}
 policyTypes:
 - Ingress
```

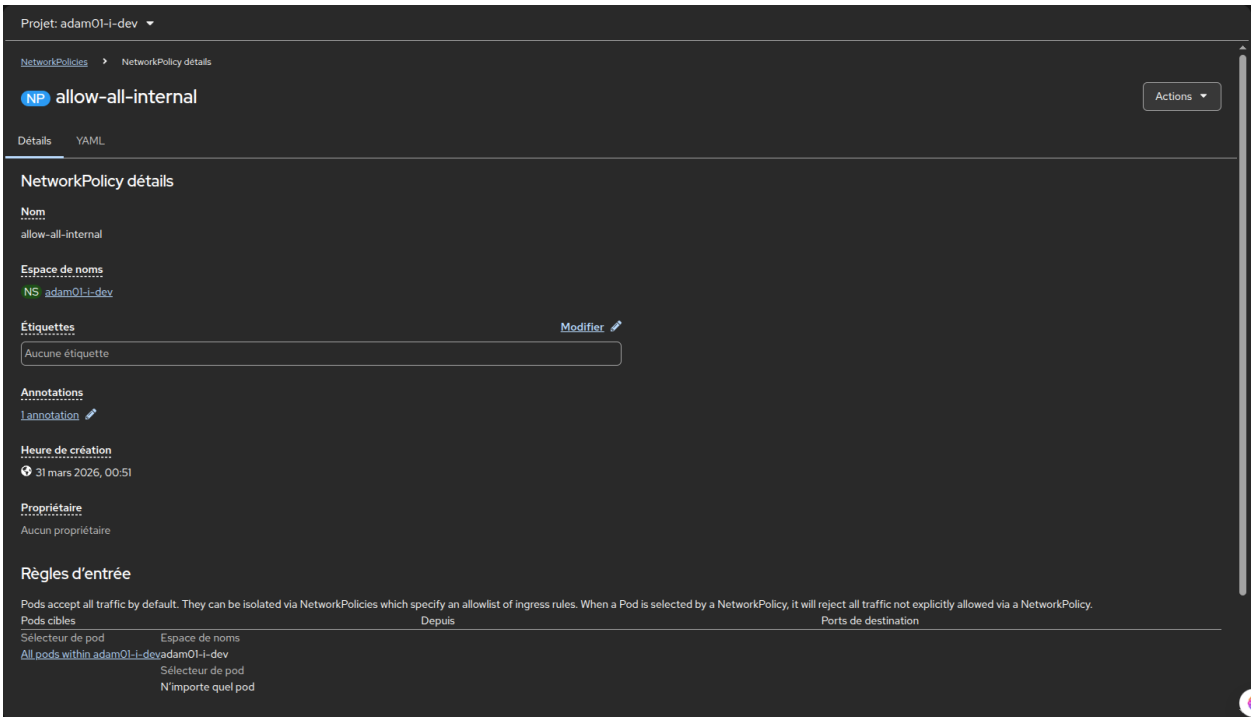

Figure 11: NetworkPolicy : allow-all-internal

b) Routes externes renvoyant 503 → correction des sélecteurs de services et ajout de `allow-from-openshift-ingress`

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
 name: allow-from-openshift-ingress
spec:
 ingress:
 - from:
 - namespaceSelector:

 matchLabels:
 kubernetes.io/metadata.name: openshift-ingress
 podSelector: {}
 policyTypes:
 - Ingress
```

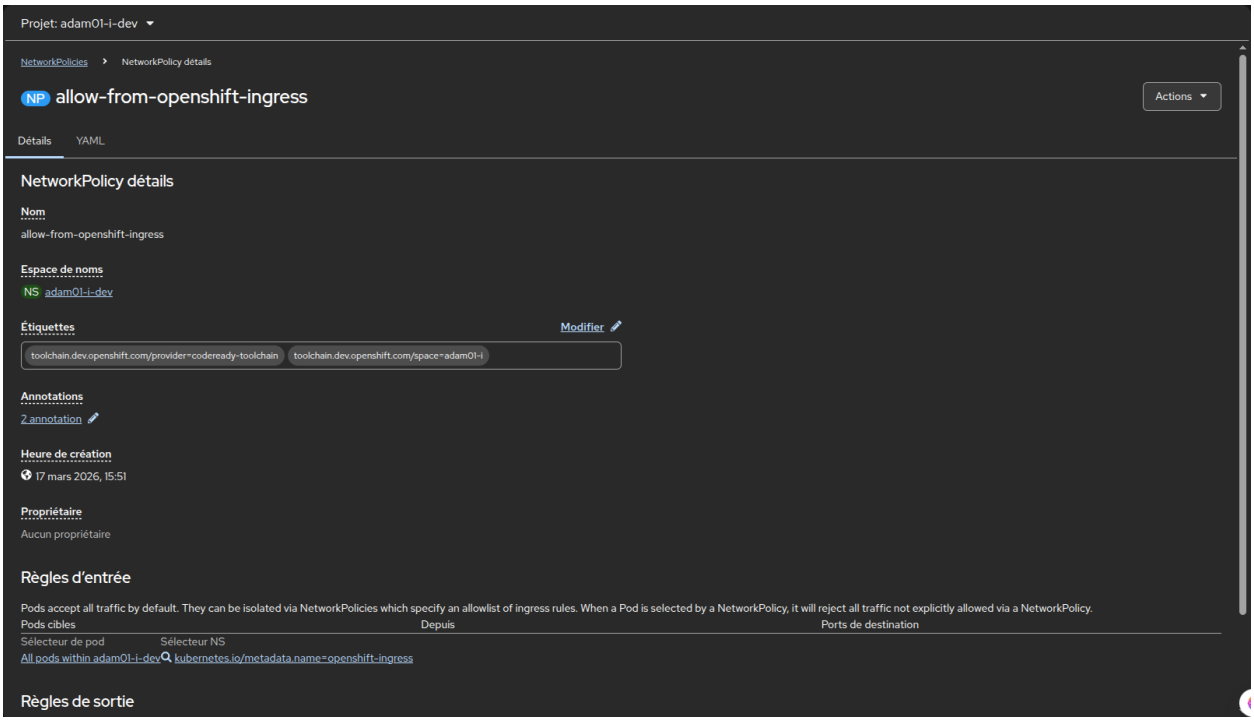

Figure 12: NetworkPolicy : allow-from-openshift-ingress

c) Dernier recours : utilisation de HTTP (sans TLS) pour les routes

Les routes ont été recréées en HTTP simple (pas de terminaison TLS) ce qui a permis de résoudre les blocages CORS et d'obtenir les réponses JSON.

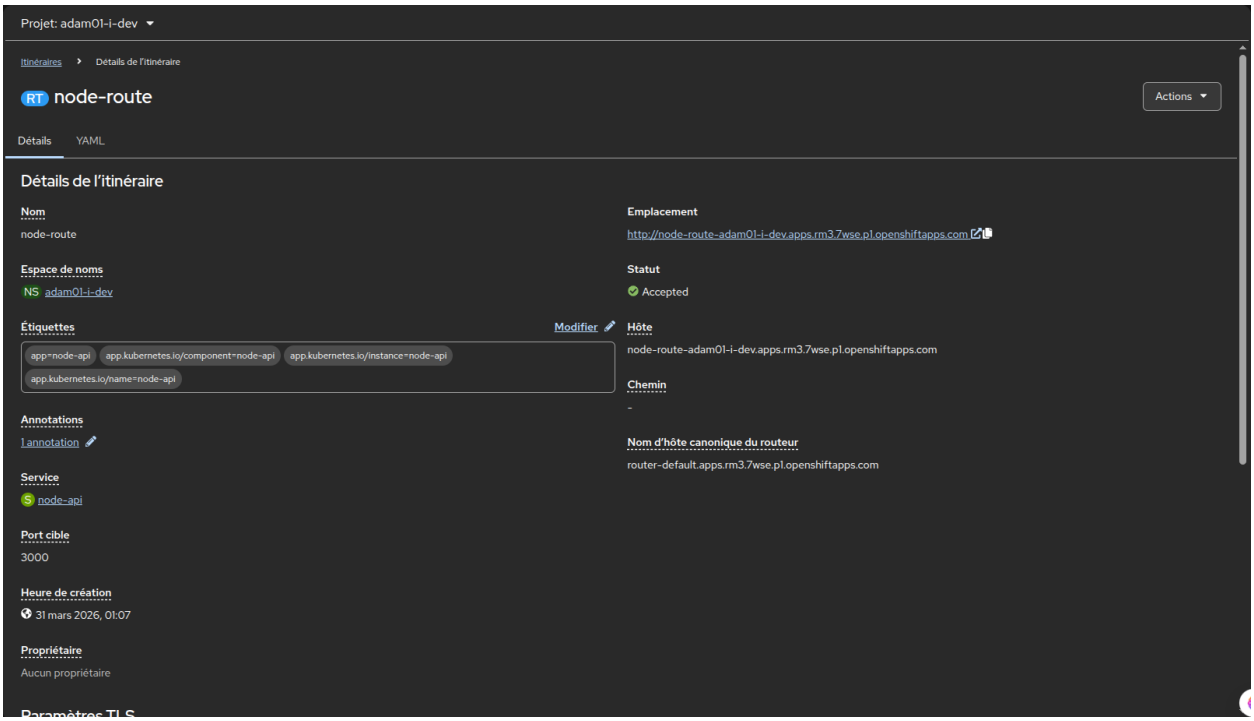

Figure 13: Utilisation de HTTP (sans TLS) pour les routes : node-route

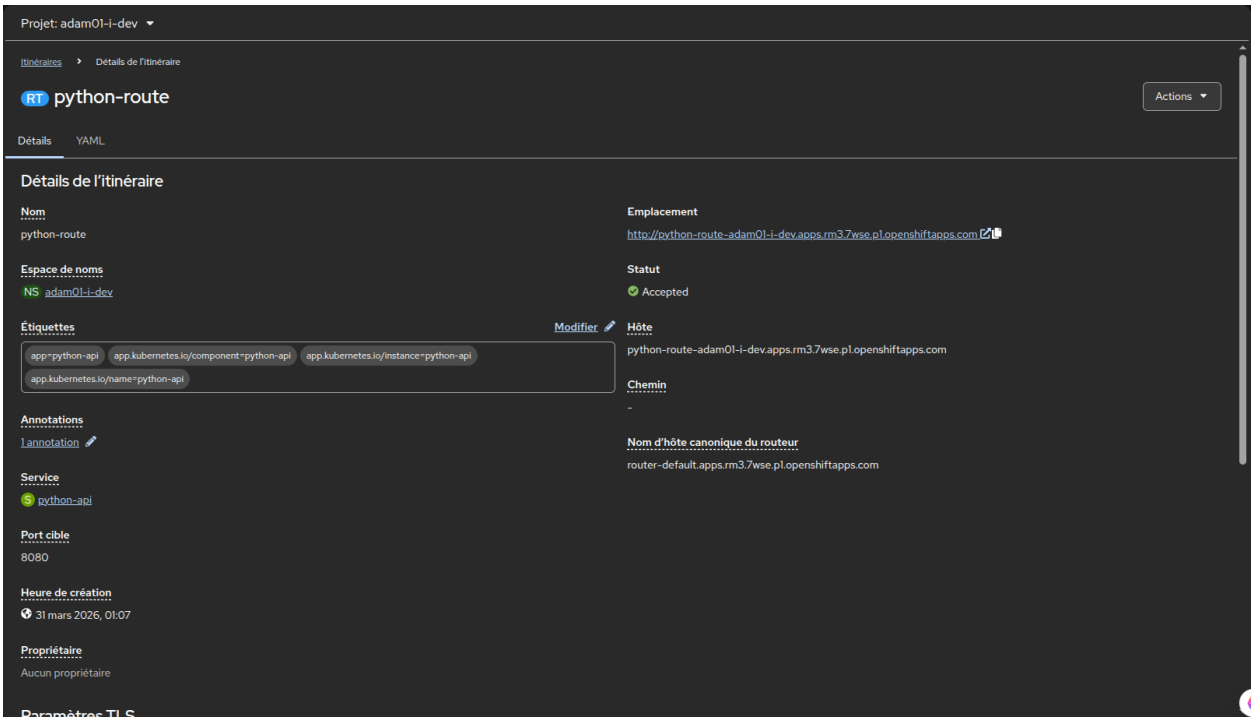

Figure 14: Utilisation de HTTP (sans TLS) pour les routes : python-route

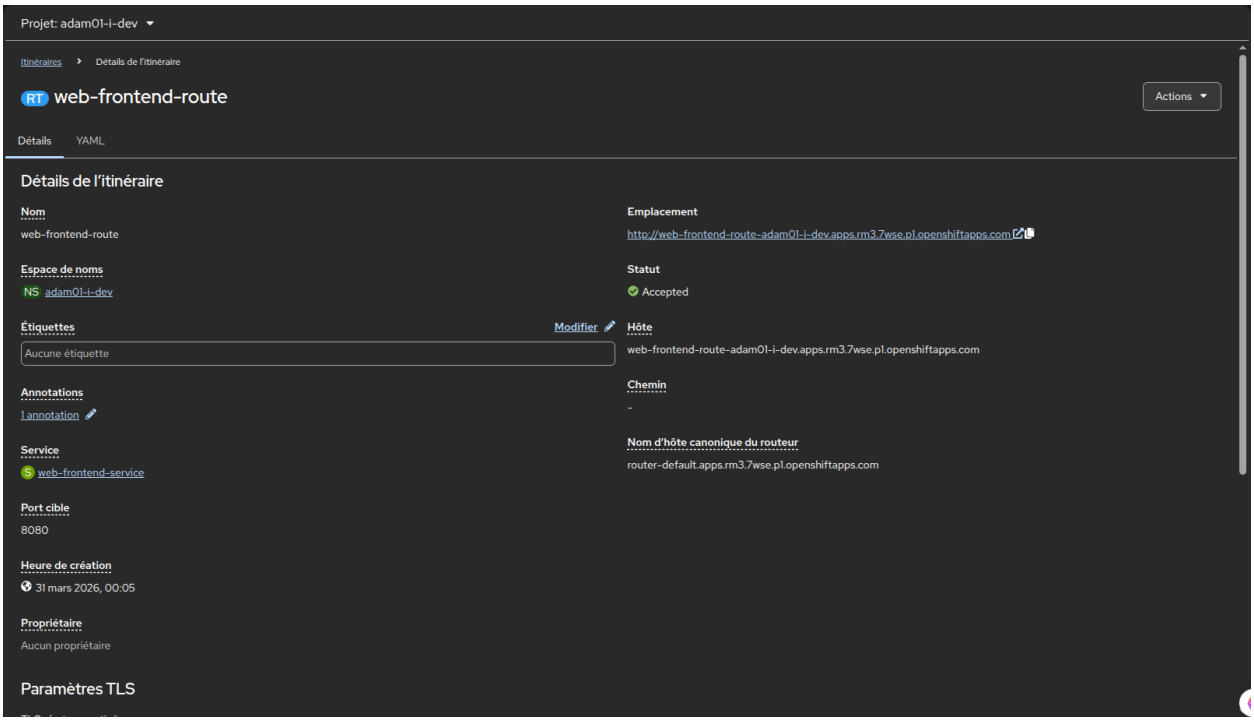

Figure 15: Utilisation de HTTP (sans TLS) pour les routes : web-frontend-route

6. Mise en place de GitHub Actions

Création d'un compte de service et d'un token OpenShift

```
oc create sa github-actions -n adam01-i-dev
oc policy add-role-to-user edit -z github-actions -n adam01-i-dev
oc create token github-actions --duration=8760h -n adam01-i-dev
```

Terminal de ligne de commande OpenShift

Terminal 4 +

```
bash-5.1 ~ $ # Vérifier que le service account existe
oc get sa github-actions -n adam01-i-dev
NAME             SECRETS   AGE
github-actions   1         165m
bash-5.1 ~ $
bash-5.1 ~ $ # Voir les détails (liens avec les secrets)
oc describe sa github-actions -n adam01-i-dev
Name:                github-actions
Namespace:           adam01-i-dev
Labels:              <none>
Annotations:         openshift.io/internal-registry-pull-secret-ref: github-actions-dockercfg-hnk9h
Image pull secrets:  github-actions-dockercfg-hnk9h
Mountable secrets:   github-actions-dockercfg-hnk9h
Tokens:              <none>
Events:              <none>
bash-5.1 ~ $
bash-5.1 ~ $ # Vérifier le rôle associé (edit)
oc get rolebinding -n adam01-i-dev | grep github-actions
bash-5.1 ~ $
```

Figure 16: Création d'un compte de service et d'un token OpenShift

Secrets GitHub

Ajoutés dans `Settings → Secrets and variables → Actions` : - `OPENSHIFT_TOKEN` : token généré - `OPENSHIFT_SERVER` : `https://api.rm3.7wse.p1.openshiftapps.com:6443` - `OPENSHIFT_NAMESPACE` : `adam01-i-dev`

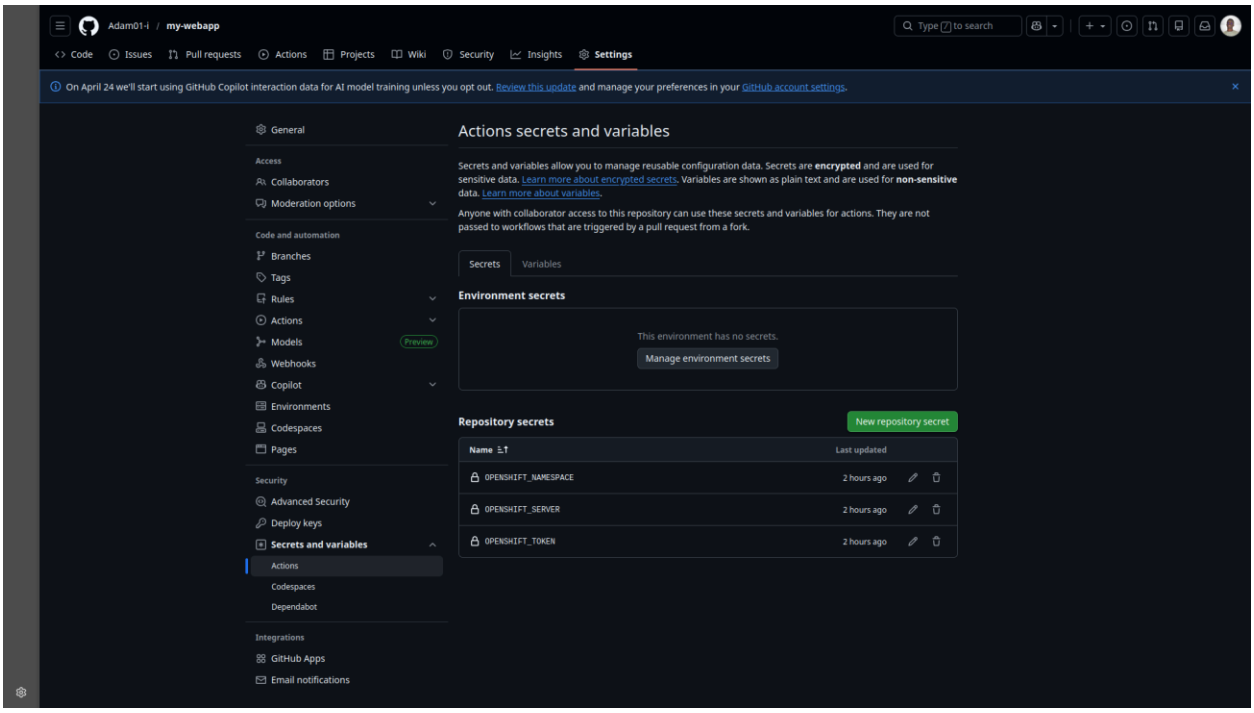

Figure 17: Secrets GitHub

```
Workflow .github/workflows/deploy.yml
name: Deploy to OpenShift
on:
  push:
    branches: [ main, master ]
jobs:
  build-nodejs:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Install OpenShift CLI
        uses: redhat-actions/openshift-tools-installer@v1
        with:
          oc: latest
      - name: Start Node.js build
        run: |
          oc login --token=${{ secrets.OPENSHIFT_TOKEN }} --server=${{
secrets.OPENSHIFT_SERVER }}
          oc start-build node-api --follow -n ${{ secrets.OPENSHIFT_NAMESPACE
}}
  build-python:
    ... (identique pour python-api)
```

```
build-frontend:
  ... (identique pour web-frontend)
```

Chaque `git push` déclenche automatiquement les trois builds.

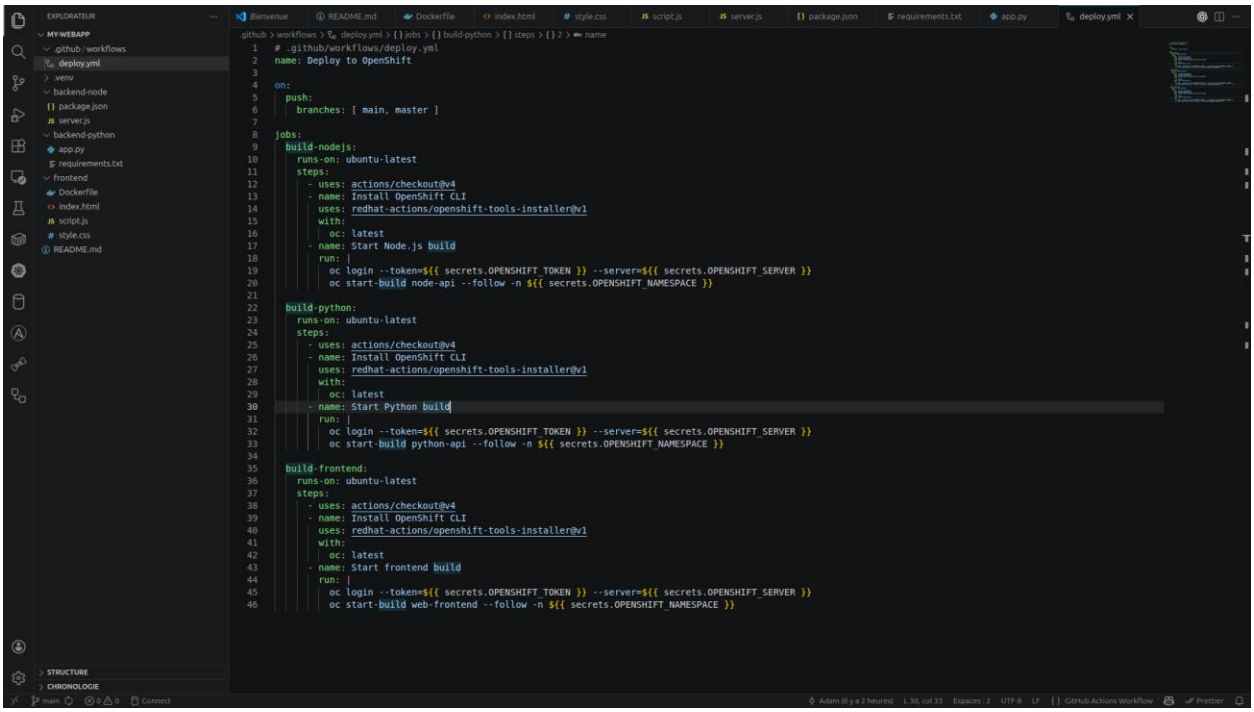

Figure 18: Workflow .github/workflows/deploy.yml

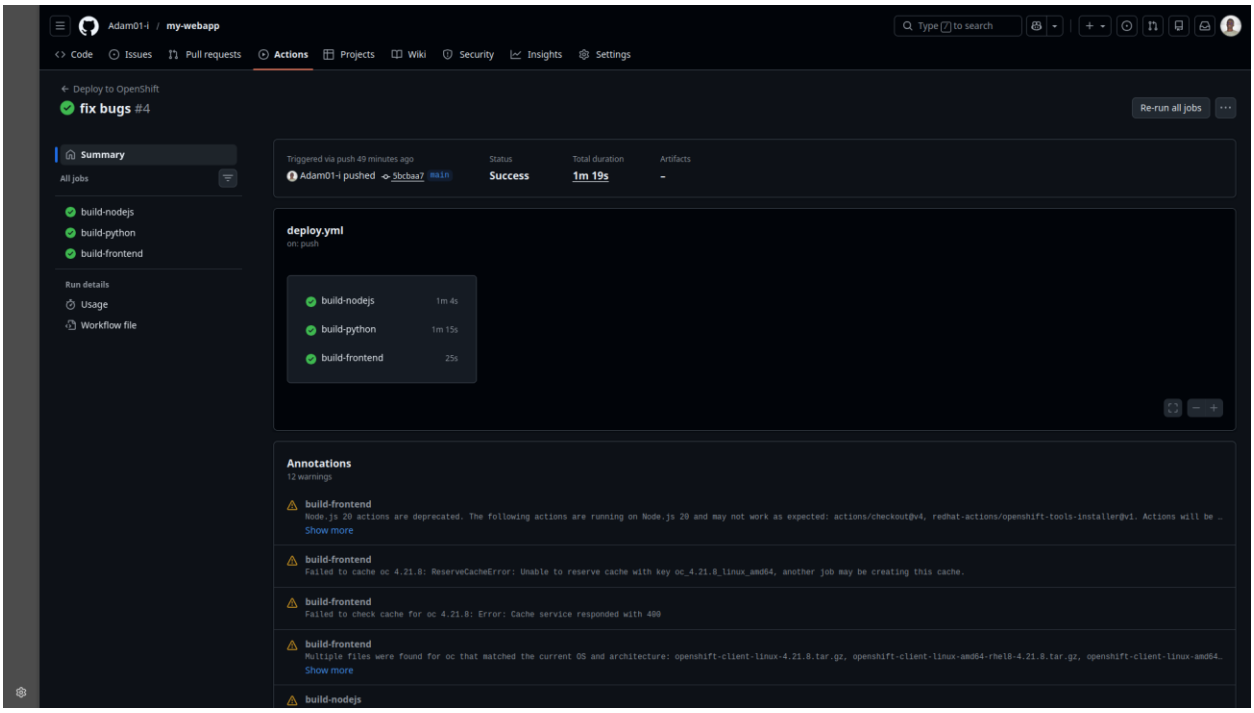

Figure 19: Resultats des builds declenches : img1

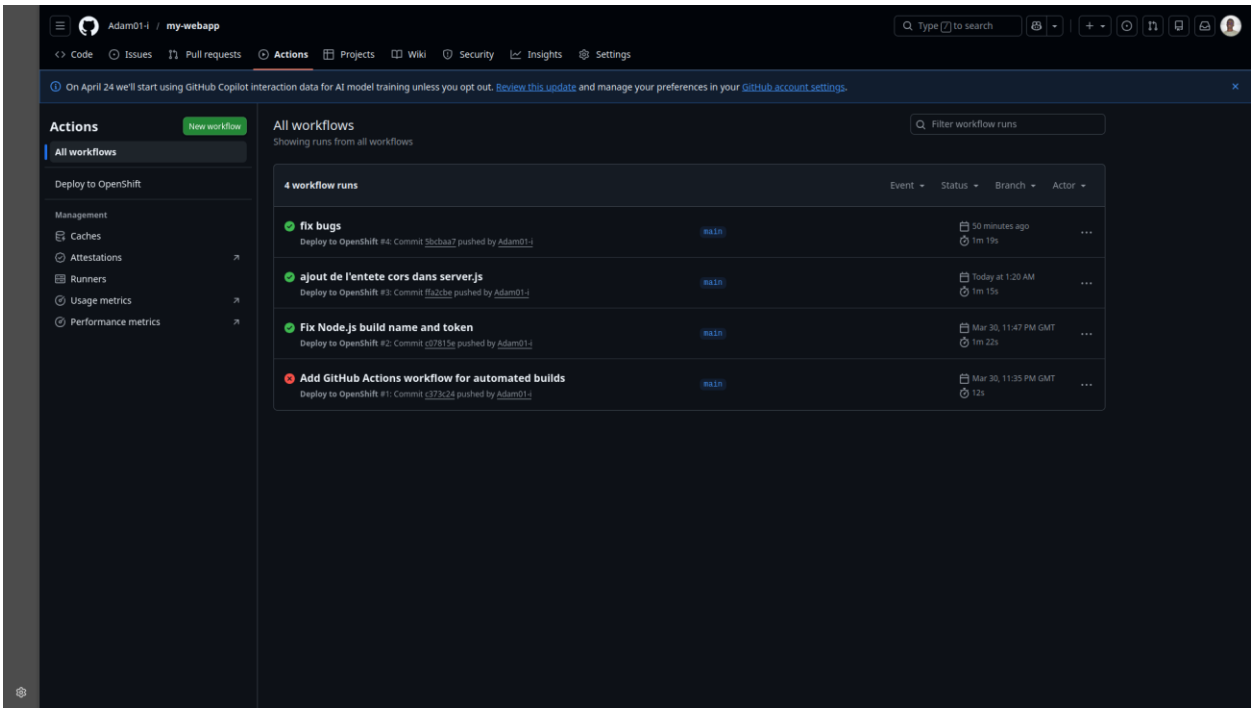

Figure 20: Resultats des builds declenches : img2

Terminal de ligne de commande OpenShift

Terminal 4 +

```
bash-5.1 ~ $ oc get builds -n adam01-i-dev
NAME             TYPE     FROM          STATUS                          STARTED            DURATION
nodejs-ex-1      Source   Git@5506d73   Complete                        13 days ago        1m16s
python-api-1     Source   Git@d10dd75   Complete                        3 hours ago        1m3s
web-frontend-2   Docker   Git@c373c24   Complete                        3 hours ago        14s
python-api-2     Source   Git@c373c24   Complete                        3 hours ago        1m1s
node-api-1       Source   Git@c373c24   Complete                        3 hours ago        47s
node-api-2       Source   Git@c07815e   Complete                        3 hours ago        58s
python-api-3     Source   Git@c07815e   Complete                        3 hours ago        1m8s
web-frontend-3   Docker   Git@c07815e   Complete                        3 hours ago        15s
web-frontend-4   Docker   Git@c07815e   Complete                        3 hours ago        18s
node-api-3       Source   Binary        Failed (InvalidContextDirectory)  About an hour ago  2s
python-api-4     Source   Binary        Failed (InvalidContextDirectory)  About an hour ago  2s
node-api-4       Source   Git@ffa2cbe   Complete                        About an hour ago  48s
python-api-5     Source   Git@ffa2cbe   Complete                        About an hour ago  58s
web-frontend-5   Docker   Git@ffa2cbe   Complete                        About an hour ago  15s
web-frontend-6   Docker   Git@5bcbaa7   Complete                        48 minutes ago     16s
python-api-6     Source   Git@5bcbaa7   Complete                        47 minutes ago     1m2s
node-api-5       Source   Git@5bcbaa7   Complete                        47 minutes ago     51s
bash-5.1 ~ $ oc get builds -n adam01-i-dev | grep node-api
node-api-1       Source   Git@c373c24   Complete                        3 hours ago        47s
node-api-2       Source   Git@c07815e   Complete                        3 hours ago        58s
node-api-3       Source   Binary        Failed (InvalidContextDirectory)  About an hour ago  2s
node-api-4       Source   Git@ffa2cbe   Complete                        About an hour ago  48s
node-api-5       Source   Git@5bcbaa7   Complete                        48 minutes ago     51s
bash-5.1 ~ $ oc get builds -n adam01-i-dev | grep python-api
python-api-1     Source   Git@d10dd75   Complete                        3 hours ago        1m3s
python-api-2     Source   Git@c373c24   Complete                        3 hours ago        1m1s
python-api-3     Source   Git@c07815e   Complete                        3 hours ago        1m8s
python-api-4     Source   Binary        Failed (InvalidContextDirectory)  About an hour ago  2s
python-api-5     Source   Git@ffa2cbe   Complete                        About an hour ago  58s
python-api-6     Source   Git@5bcbaa7   Complete                        48 minutes ago     1m2s
bash-5.1 ~ $ oc get builds -n adam01-i-dev | grep web-frontend
web-frontend-2   Docker   Git@c373c24   Complete                        3 hours ago        14s
web-frontend-3   Docker   Git@c07815e   Complete                        3 hours ago        15s
web-frontend-4   Docker   Git@c07815e   Complete                        3 hours ago        18s
web-frontend-5   Docker   Git@ffa2cbe   Complete                        About an hour ago  15s
web-frontend-6   Docker   Git@5bcbaa7   Complete                        48 minutes ago     16s
bash-5.1 ~ $
```

Figure 21: Resultats des builds declenches : img3

Problèmes rencontrés et solutions

Problème	Solution
VMs KubeVirt s'arrêtent aléatoirement	Abandon des VMs, utilisation de pods standard
Pod PHP ne parvient pas à écouter sur le port 80	Remplacement par une API Python (Flask)
Image `nginx:alpine` non-root impossible à écrire dans `/var/cache/nginx`	Utilisation de `nginxinc/nginx-unprivileged`

Connexion MySQL impossible (timeout)	Ajout de NetworkPolicy `allow-all-internal`
Routes externes renvoient 503	Correction des sélecteurs des services (`deployment=node-api`), ajout de `allow-from-openshift-ingress`
CORS bloqué (pas d'en-tête `Access-Control-Allow-Origin`)	Ajout manuel dans Node.js et `flask_cors` dans Python
Webhooks GitHub en échec (403)	Passage à GitHub Actions avec token

État final fonctionnel

- **Route Node.js** (HTTP) : `http://node-route-adam01-i-dev.apps.rm3.7wse.p1.openshiftapps.com`
  - Retourne : `{"message":"Connexion Node.js -> MySQL OK"}`
- **Route Python** (HTTP) : `http://python-route-adam01-i-dev.apps.rm3.7wse.p1.openshiftapps.com`
  - Retourne : `{"message":"Connexion Python -> MySQL OK"}`
- **Route Frontend** (HTTP) : `http://web-frontend-route-adam01-i-dev.apps.rm3.7wse.p1.openshiftapps.com`
  - Affiche un tableau de bord avec les deux messages de succès.

Capture d'écran des tests `curl` :

```
$ curl http://node-route-adam01-i-dev.apps.rm3.7wse.p1.openshiftapps.com
{"message":"Connexion Node.js -> MySQL OK"}

$ curl http://python-route-adam01-i-dev.apps.rm3.7wse.p1.openshiftapps.com
{"message":"Connexion Python -> MySQL OK"}
```

Terminal de ligne de commande OpenShift

Terminal 4 +

```
bash-5.1 ~ $ curl http://node-route-adam01-i-dev.apps.rm3.7wse.p1.openshiftapps.com
{"message":"Connexion Node.js -> MySQL OK"}bash-5.1 ~ $
bash-5.1 ~ $
bash-5.1 ~ $ curl http://python-route-adam01-i-dev.apps.rm3.7wse.p1.openshiftapps.com
{"message":"Connexion Python -> MySQL OK"}
bash-5.1 ~ $
bash-5.1 ~ $ curl http://web-frontend-route-adam01-i-dev.apps.rm3.7wse.p1.openshiftapps.com
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Mon App - MySQL + Node.js + Python</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="container">
        <h1>📊Tableau de bord des APIs</h1>
        <div class="cards">
            <div class="card node">
                <h2>🟢API Node.js</h2>
                <p id="node-status">Chargement...</p>
                <pre id="node-data"></pre>
            </div>
            <div class="card python">
                <h2>🐍API Python</h2>
                <p id="python-status">Chargement...</p>
                <pre id="python-data"></pre>
            </div>
        </div>
        <div class="footer">
            <p>✨Données provenant de MySQL (persistant) ✨/p>
        </div>
    </div>
    <script src="script.js"></script>
</body>
</html>
bash-5.1 ~ $
```

Figure 22: Tests curl : routes : python, node, frontend

Interface web :

Tableau de bord : (description: Le tableau de bord montre deux cartes “API Node.js” et “API Python” avec le statut “✅Connecté” et les JSON respectifs. Un pied de page indique “Données provenant de MySQL (persistant)”.).

Lien public de l’application : **http://web-frontend-route-adam01-i-dev.apps.rm3.7wse.p1.openshiftapps.com/**

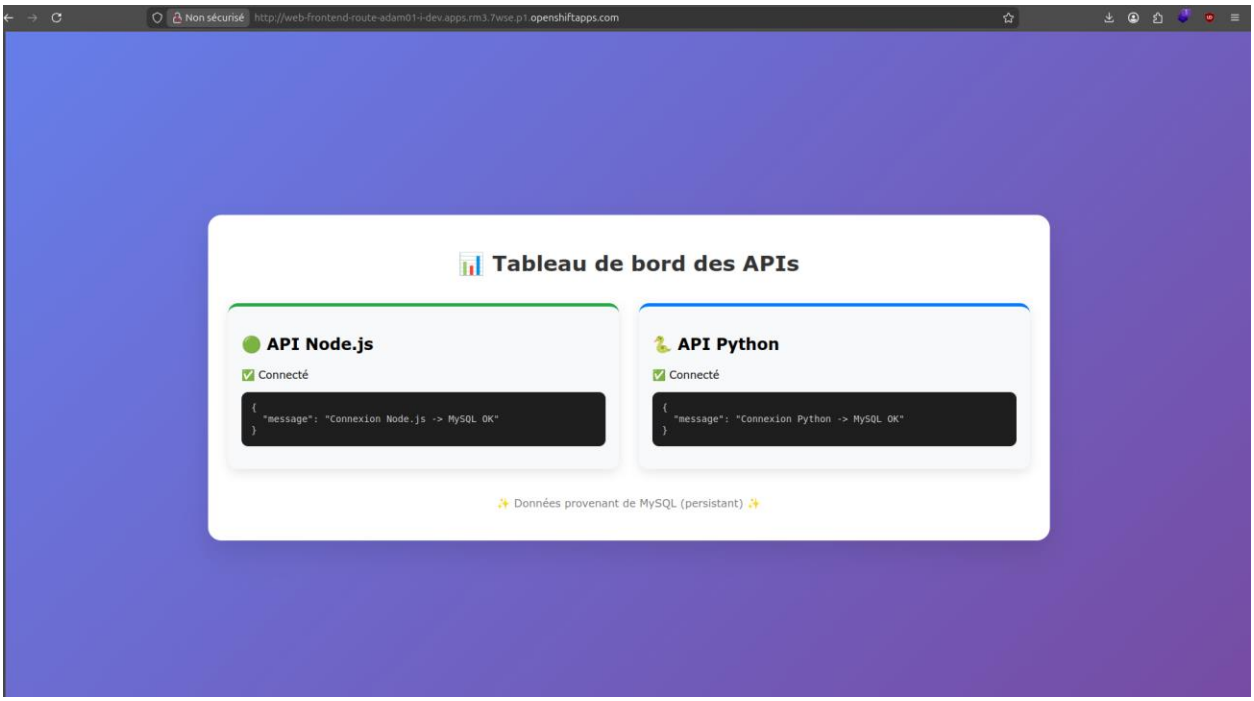

Figure 23: Resultat du lien public de l'app via Interface web

Automatisation CI/CD (GitHub Actions)

Le dépôt GitHub : **`https://github.com/Adam01-i/my-webapp`** contient le code source organisé ainsi :

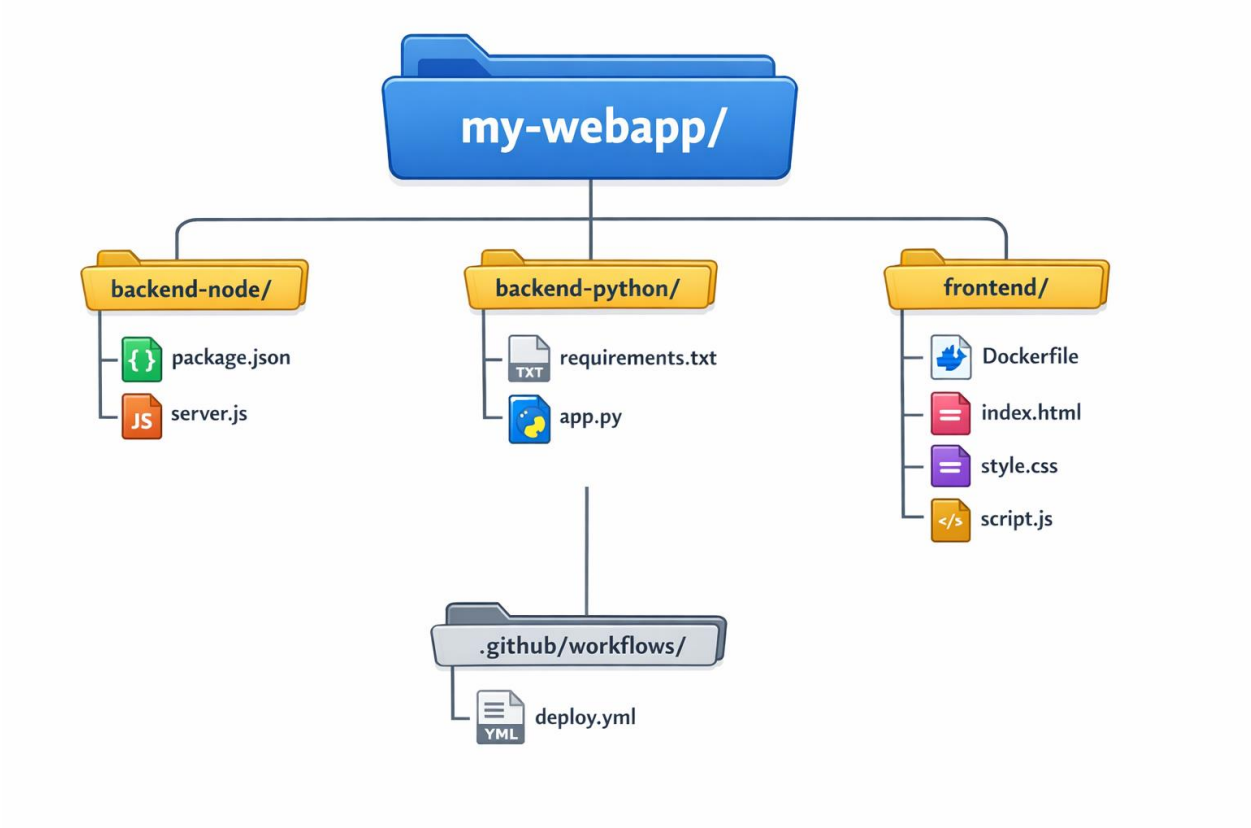

Figure 24: Structure du code sur github

À chaque `git push` sur la branche `main`, le workflow s'exécute et déclenche les builds S2I d'OpenShift pour les trois services. Les nouvelles images sont automatiquement déployées (le déploiement est géré par OpenShift via les `ImageStream` et les `Deployment`).

Vérification des builds :

```
$ oc get builds
NAME              TYPE     STATUS
node-api-1        Source   Complete
python-api-1      Source   Complete
web-frontend-1    Source   Complete
```

Terminal de ligne de commande OpenShift

Terminal 4 +

```
bash-5.1 ~ $ oc get builds
NAME             TYPE     FROM          STATUS                          STARTED            DURATION
nodejs-ex-1      Source   Git@5506d73   Complete                        13 days ago        1m16s
python-api-1     Source   Git@d10dd75   Complete                        3 hours ago        1m3s
web-frontend-2   Docker   Git@c373c24   Complete                        3 hours ago        14s
python-api-2     Source   Git@c373c24   Complete                        3 hours ago        1m1s
node-api-1       Source   Git@c373c24   Complete                        3 hours ago        47s
node-api-2       Source   Git@c07815e   Complete                        3 hours ago        58s
python-api-3     Source   Git@c07815e   Complete                        3 hours ago        1m8s
web-frontend-3   Docker   Git@c07815e   Complete                        3 hours ago        15s
web-frontend-4   Docker   Git@c07815e   Complete                        3 hours ago        18s
node-api-3       Source   Binary        Failed (InvalidContextDirectory)  About an hour ago  2s
python-api-4     Source   Binary        Failed (InvalidContextDirectory)  About an hour ago  2s
node-api-4       Source   Git@ffa2cbe   Complete                        About an hour ago  48s
python-api-5     Source   Git@ffa2cbe   Complete                        About an hour ago  58s
web-frontend-5   Docker   Git@ffa2cbe   Complete                        About an hour ago  15s
web-frontend-6   Docker   Git@5bcbaa7   Complete                        56 minutes ago     16s
python-api-6     Source   Git@5bcbaa7   Complete                        56 minutes ago     1m2s
node-api-5       Source   Git@5bcbaa7   Complete                        56 minutes ago     51s
bash-5.1 ~ $
```

Figure 25: Resultats des builds apres un push qui declenche un deploiement automatique

Commandes utiles (annexe)

Gestion des ressources

```
oc get pods -n adam01-i-dev
oc get svc -n adam01-i-dev
oc get routes -n adam01-i-dev
oc get endpoints
oc get configmap
oc get deployment
```

Redémarrage et logs

```
oc rollout restart deployment/<nom>
oc logs deployment/<nom> --tail=50
oc describe pod <pod-name>
```

NetworkPolicy

```
oc apply -f networkpolicy.yaml
oc get networkpolicy
```

Port-forward (pour test local)

```
oc port-forward service/node-api 3001:3000 &
curl http://localhost:3001
```

GitHub Actions – mise à jour du token

```
oc create token github-actions --duration=8760h -n adam01-i-dev
```

Conclusion

Le projet a atteint tous ses objectifs :

- **Application web fonctionnelle** avec base de données MySQL persistante.
- **Deux APIs (Node.js et Python)** accessibles via routes HTTP.
- **Frontend statique** interrogeant les APIs et affichant les résultats.
- **CI/CD complet** via GitHub Actions (build et déploiement automatiques à chaque `git push`).
- **Résolution de nombreux problèmes** : stabilité, permissions, CORS, NetworkPolicies, etc.

Lien gituhp du projet : https://github.com/Adam01-i/my-webapp

Lien public de l'application : http://web-frontend-route-adam01-i-dev.apps.rm3.7wse.p1.openshiftapps.com/